Operating system security – Lab

CS930 Foundations of Cyber Security

University of Warwick

Work individually, completing the tasks in order. Apply the lecture's Unix permissions ideas to this fictional workspace. For each task, inspect the relevant objects, predict the results, choose your commands, then test your reasoning. Use the Linux command reference on pages 5–6; you do not need to memorise the syntax. Keep any brief notes you find useful outside the guest. Worked answers will be released later for self-checking; no formal report is required.

Before starting. Open `https://warwick-cs930.github.io/labs/lecture-03/`. Wait for `CHECK PASSED`, `SETUP COMPLETE`, and `LAB READY`. Use `id` to check that your UID is nonzero. Keep a copy of the exact `LAB_DIRECTORY:` path and check it against `pwd`. If either check fails, retain the output and ask for help. Work only inside this disposable directory; keep the original contents of existing files.

Task 1: Implement a file policy

The course team needs two successive policies for `practice.txt`:

- **Review:** the owner may read, but nobody receives write or execute permission. Group and other receive no permissions.
- **Editing:** the owner may read and append. Group and other still receive no permissions; the file remains non-executable.

1. Inspect the file's numerical owner and current permissions. Compare the owner with your account and identify which grants conflict with the review policy.
2. Derive the review mode before applying it. Use one `chmod` command, then test both a read and an append. Check that a denied edit leaves the original text intact.
3. Move to the editing policy by adding only the permission now needed. Test both reading and appending again.

Check your understanding: How do the initial and chosen modes explain the read/append outcomes for each policy? Why can the owner change a file's permissions while its contents are read-only?

Task 2: Diagnose an unexpected denial

Inspect `owner-puzzle.txt` and predict whether you can read it. Test your prediction before making changes. Two suggested explanations are:

> "There is a read bit somewhere in the mode, so the owner can read it."
> "Starting another `cat` process gives it a new PID, so it may gain access."

Use the ownership and mode to evaluate both claims. Then repair the file so its owner can read and append, with no execute bit or group/other grants. Preserve its contents and demonstrate both permitted operations.

Check your understanding: Did the read result match your prediction? Which permission class explains it, and why does your repair meet the policy? Explain whether a new PID under the same account would change the decision.

Task 3: Design private creation defaults

New notes must be readable and writable only by their owner. For new directories, only the owner may list names, traverse the directory, and create or remove entries. Text files must not be executable. Choose the **least restrictive creation mask** that meets these requirements.

With no inherited default ACL, ordinary files request `666` and directories request `777`. A mask removes requested bits: `result = requested & ~mask`.

1. Inspect `private-notes.txt`. Derive your mask and predict the two new modes. Predict whether setting the mask will change the existing file's mode.
2. Use one subshell to set your chosen mask and create `new-private.txt` with a short note and an empty `new-private-dir`. Choose commands from the reference. Do not use `chmod` on these two new objects: their initial modes must meet the policy.
3. Inspect all three objects and compare their modes with your predictions. Bring the existing notes into line with the same file policy, preserving their contents. Test reading and appending to both notes files.

Check your understanding: Do the observed modes match your mask calculation? Why does the existing file need separate treatment? Explain why one creation mask gives different modes to a regular file and a directory.

Task 4: Investigate names, links, and deletion

The archive contains an original `report.txt` and a labelled `disposable-copy.txt`. Keep the original report untouched throughout this task.

1. Inspect both files' contents and inode numbers. Are identical contents sufficient evidence that two names refer to the same file?
2. In the lab root, create `duplicate-hard.txt` as a hard link to the disposable copy and `duplicate-soft.txt` as a symbolic link to `archive/disposable-copy.txt`. Inspect all three names and check that both new links can be read.
3. Inspect the archive directory and copy's permissions. Predict whether removing the copy's original name will work, which remaining names will still be readable, and how the hard-link count will change.

Test the prediction by removing **only** the labelled copy's name:

```
rm -f archive/disposable-copy.txt
```

Inspect and read both remaining links. Leave them in place for the final audit; leave the archive directory's mode unchanged for now.

Check your understanding: How did the inode numbers, link count, and read results compare with your predictions? Explain which object's permissions controlled removal and why the links behave as observed.

Task 5: Restore a broken editing workflow

An editor cannot use `materials/week-03/briefing.txt` as required. Diagnose the problem from evidence and make the smallest permission changes that satisfy all of these requirements:

- The owner can read the briefing and append a short release decision without losing the original text.
- Both directories in its path remain listable and traversable by the owner.
- Group and other have no grants on these objects; the text file is non-executable.

First reproduce the failed read and append. Decide what to inspect next. Change **one object at a time**, testing both reading and appending after each change before making another. Do not replace files, change ownership, or apply a recursive permission change.

Check your understanding: Trace the read/append results after each change. Identify the object responsible for each denial. Explain why a successful read alone is insufficient evidence that the workflow is repaired.

Task 6: Enforce an archive policy

The archive must support this combination of operations:

- List its entries and read `report.txt`; preserve the report's original contents and mode.
- Read and directly append to the existing `working-notes.txt`.
- Deny creation and removal of entries inside the archive.
- Give group and other no permissions; do not make text files executable.

A proposed fix is `chmod -R 500 archive`. **Evaluate it on paper only**: name a required operation it would break and an unnecessary grant it would introduce. Then design and apply your own changes using the fewest permission-bit changes needed.

Write a test plan with a command and expected result for each required operation. Include successful reads and a note append, an attempted creation of `archive/new.txt`, and an attempted removal of `archive/report.txt`.

Before the removal test, inspect the archive's final mode and explain why it should deny removal. If unsure, ask for help before proceeding. Only then attempt `rm -f archive/report.txt`, and check that the original report remains readable and unchanged. If removal succeeds, retain the failure evidence and ask for help; recreating the report does not demonstrate protection. Run the other tests and compare every outcome with your plan.

Check your understanding: Do all test results support your chosen policy? Were the faults unsuitable settings or broken permission checks? Can these settings prevent the owning account from deliberately undoing the freeze? Explain.

Task 7: Check your results and clean up

Before finishing, check these outcomes and keep any notes you want to compare with the worked answers:

- The practice, private, owner-puzzle, and briefing files meet their owner-access requirements.
- The states of both links agree with your explanation of names, inodes, and target paths.
- The archive permits reading and direct note appends, while denying entry creation and removal.

A few predictions, commands, and explanations are enough for self-checking; a full transcript is unnecessary. What can you conclude from experiments with one ordinary account? Does a file's broad mode alone prove another account could read it through a private parent directory?

Run `pwd`. Continue only if it matches the exact lab path you kept before starting. Restore directory access and remove only this attempt's named files:

```
chmod u+rwx materials materials/week-03 archive
rm -f practice.txt owner-puzzle.txt private-notes.txt new-private.txt
rm -f duplicate-hard.txt duplicate-soft.txt
rm -f materials/week-03/briefing.txt
rm -f archive/report.txt archive/disposable-copy.txt
rm -f archive/working-notes.txt archive/new.txt
rmdir materials/week-03 materials archive
rmdir new-private-dir
cd ..
```

Omit `rmdir new-private-dir` if you did not create it. The `-f` option ignores missing files. Finally use `rmdir` with the **exact lab-directory name from the end of your saved path**. If it is nonempty, inspect what remains with the instructor; do not use wildcards or recursive deletion.

Linux command reference

Use this reference throughout the tasks. Uppercase words such as `FILE`, `DIR`, `PATH`, and `MODE` are placeholders: replace them with the required name or value. Do not type a prompt such as `$`. Run one command per line and press Enter. Use the ordinary account supplied by the lab.

Navigate and inspect

A path such as `archive/report.txt` is relative to your current directory. A path starting with `/` is absolute. `.` means the current directory; `..` means its parent. Keep the shell in the lab root during the tasks, so the supplied paths resolve correctly.

Command	Meaning
`pwd`	Print the current directory's absolute path.
`id`	Show your user ID (UID), primary group, and group memberships.
`cd DIR`; `cd ..`	Change directory; move to the parent directory.
`ls DIR`	List names inside a directory.
`ls -l PATH`	Long listing: type/permissions, hard-link count, owner, group, size, date, and name. For a directory, list its entries.
`ls -ld DIR`	Inspect the directory itself, rather than its contents.
`ls -ln FILE`	Long listing with numerical owner and group IDs, for comparison with `id`.
`ls -li FILE`	Long listing with the inode number added at the start. Compare inode numbers and hard-link counts.
`cat FILE`	Display a file's contents. A failed read prints an error.

Options can be combined: `ls -ldn DIR` inspects a directory with numerical IDs. Commands such as `ls -li FILE1 FILE2` and `cat FILE1 FILE2` accept several paths separated by spaces. So do `chmod MODE PATH1 PATH2`, `rm -f FILE1 FILE2`, and `rmdir DIR1 DIR2`. A symbolic link's long listing includes `NAME -> TARGET`.

Read and change permission bits

In a listing such as `-rw-r-----`, the first character is the object type (`-` for a regular file, `d` for a directory, `l` for a symbolic link). The next three groups of three are owner, group, and other. Each numerical digit adds the granted bits: `r = 4`, `w = 2`, `x = 1`; `-` contributes zero. For example, `r-x` is 5.

Command	Meaning
`chmod MODE PATH`	Set all three ordinary permission classes using a three-digit octal mode. Choose the digits to match the policy.
`chmod u+w FILE`	Add owner write permission; preserve the other bits.
`chmod u-w FILE`	Remove owner write permission; preserve the other bits.
`chmod u+x DIR`	Add owner directory-search permission.
`chmod g=r FILE`	Set the group class to read only, replacing its existing grants.
`chmod go-rwx PATH`	Remove all group and other permissions.

Symbolic modes use `u` (owner), `g` (group), `o` (other), or `a` (all classes), followed by `+`, `-`, or `=`. Several permission letters can follow one operator: `u+rw` adds owner read and write. Combine changes with commas, for example `chmod u+r,g-w FILE`. `chmod -R` applies changes recursively; it appears in Task 6 only as a proposal to evaluate, not a command to run.

Linux command reference (continued)

Create files and directories

`printf` prints text. The shell's `>` and `>>` operators send that text to a file. Quotes keep the message together; `\n` inside the format string adds a newline.

Command or shell syntax	Meaning
`printf 'A short note\n'`	Print a message and a newline in the terminal.
`printf 'A note\n' > NEWFILE`	Create a file containing the message. **If it exists, `>` truncates its old contents.** Use this only for a named new file.
`printf 'An edit\n' >> FILE`	Append a message, keeping existing contents. Read the file afterward to check the result.
`touch NEWFILE`	Create an empty file if the name is absent. If it exists, update its timestamps without erasing the text.
`mkdir DIR`	Create a directory.
`umask`	Display the current creation mask.
`umask MASK`	Set the mask for subsequent creations by this shell and its child processes. Existing objects keep their modes.

An append can also create a file when its name is absent. Check the spelling of paths; use only the names requested by the tasks. An expected "Permission denied" result is part of an experiment, not an instruction to switch accounts.

Use a temporary mask in a subshell

Parentheses run commands in a subshell. Its mask changes do not alter the surrounding shell. Replace `MASK`, `NEWFILE`, and `NEWDIR` below with your choices. The placeholder `MASK` is not a value to type literally.

```
(
    umask MASK
    printf 'A new note\n' > NEWFILE
    mkdir NEWDIR
)
```

The shell may display a continuation prompt until you enter the closing parenthesis. Finish the block before inspecting the results. On one line, semicolons separate commands: `( umask MASK; touch NEWFILE )`.

Create links and remove names

Command	Meaning
`ln SOURCE NEWNAME`	Create a hard link: another name for the same file.
`ln -s TARGET NEWNAME`	Create a symbolic link storing `TARGET`. A relative target is interpreted from the directory containing the link.
`rm -f FILE`	Remove a named file or link. `-f` suppresses prompts and missing-name errors; it does not override permissions. Removing a symlink removes the link, not its target.
`rmdir DIR`	Remove an empty directory. A nonempty directory is left intact.

Use only the explicitly named disposable objects. For cleanup, follow Task 7; do not use wildcards or recursive deletion. Run experiments once in a fresh lab session, keeping useful notes outside the guest before restarting.